

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 11 | Queues: Linear Queue, Circular Queue, Deque and Priority Queue

Week 11 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 11 at a glance

Item	Details
Lab title	Queues: Linear Queue, Circular Queue, Deque and Priority Queue
Week	11 of 16
Prerequisites	Lab 09 (Circular Linked Lists), Lab 10 (Stacks)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab11_Instructor.ipynb , Lab11_Student.ipynb
Next lab	Lab 12: Hashing and Hash Tables

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the **queue ADT** and the **FIFO** principle.
2. Implement `enqueue`, `dequeue`, `front/peek`, `rear`, `is_empty`, `is_full` in $O(1)$.
3. Explain why a simple array queue wastes space and fix it with a **circular queue** (modulo arithmetic).
4. Implement a **linked-list queue** and use Python's `collections.deque`.
5. Use a **deque** (double-ended queue) and solve the **sliding window maximum** problem.
6. Understand **priority queues** and use Python's `heapq` (details in Lab 15).
7. Simulate real systems: ticket counters, printers and CPU scheduling.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain queue variants as abstract data types	Understand (C2)
CLO-2	Implement queues, circular queues and deques	Apply (C3)
CLO-3	Apply queues to simulations and algorithms	Apply / Analyse (C3–C4)

2. Motivation: First Come, First Served

Whenever people or tasks wait for a resource, fairness demands that **whoever came first is served first**. Think of the line at a bank, a fee-submission counter or a petrol pump. Computers face the same situation constantly: print jobs, network packets, customer orders and messages all wait in **queues**.

Analogy: the fee counter at the university

Students join the line at the **rear** and are served at the **front**. Nobody jumps the line. First In, First Out: **FIFO**. The difference from a stack: a stack removes the **newest** item; a queue removes the **oldest**.

2.1 Queues in Daily Life

Example	Type of queue
Bank, ticket or fee counter line	Simple queue (FIFO)
Hospital emergency ward (serious patients first)	Priority queue
Call centre: “your call is number 5 in the queue”	Simple queue
Foodpanda / restaurant orders in the kitchen	Queue (sometimes priority)
Traffic at a signal	Queue
Tasbeeh counting beads going round	Circular queue

2.2 Queues in Industry

- **Operating systems:** ready queues for CPU scheduling, printer spooling, keyboard buffers.
- **Networking:** routers store packets in queues; video streaming uses buffers (circular queues).
- **Message queues** (RabbitMQ, Apache Kafka, AWS SQS) connect services in large systems like Daraz, Careem and banks; orders and payments are processed through queues.
- **Web servers** queue incoming requests; **task queues** (Celery) run background jobs.
- **Breadth-First Search** (Lab 16), used in maps and social networks, is driven by a queue.

2.3 Queues in Research

- **Queueing theory** (a branch of mathematics) models waiting times in hospitals, telecom networks and traffic systems.
- **Reinforcement learning:** experience replay buffers (fixed-size deques) store recent experiences for training agents.
- **Real-time AI on sensor streams** (e.g., activity recognition) keeps the last N readings in a circular buffer.

🌟 2.4 Why It Matters in 2026–2030

💡 The world runs on asynchronous systems

Modern apps are made of many services that talk through **queues**: an order is placed, queued, processed by payment, then by delivery. AI systems queue thousands of user requests for GPUs. Understanding queues, buffers and priorities is essential for cloud, distributed systems and AI engineering.

📖 3. Part A: The Queue ADT

Operation	Meaning	Time
<code>enqueue(x)</code>	Add x at the rear	$O(1)$
<code>dequeue()</code>	Remove and return the front element	$O(1)$
<code>front()</code> / <code>peek()</code>	Return the front element without removing it	$O(1)$
<code>rear()</code>	Return the last element without removing it	$O(1)$
<code>is_empty()</code> / <code>is_full()</code>	Check the state	$O(1)$
<code>size()</code>	Number of elements	$O(1)$

FIFO in action

```

enqueue(10) enqueue(20) enqueue(30)      dequeue() -> 10
front                rear                front      rear
|                    |                    |         |
[10] -> [10, 20] -> [10, 20, 30] -> [20, 30]

```

3.1 Why Not Just Use a Python List?

`list.append(x)` is $O(1)$, but `list.pop(0)` must **shift all remaining elements left**, so it is $O(n)$. For a queue with a million items, every dequeue would move a million elements.

```

import time
from collections import deque

n = 100_000
lst = list(range(n))
t = time.perf_counter()
while lst:
    lst.pop(0)          # O(n) each
t_list = time.perf_counter() - t

dq = deque(range(n))
t = time.perf_counter()
while dq:
    dq.popleft()        # O(1) each
t_deque = time.perf_counter() - t
print(f"list.pop(0): {t_list:.3f}s    deque.popleft(): {t_deque:.4f}s")

```

4. Part B: Circular Queue (Array-Based)

With a fixed array, a simple queue moves **front** forward on every dequeue. Soon **rear** reaches the end of the array even though there is **empty space at the beginning**. A **circular queue** solves this by **wrapping around**: after the last index comes index 0. We use the **modulo** operator: **next = (i + 1) % capacity**.

Wrap-around

capacity = 5, after enqueue A B C D, dequeue A B, enqueue E F

index:	0	1	2	3	4
	[F]	[]	[C]	[D]	[E]
	^		^		
	rear		front		

F wrapped around to index 0!

```

class CircularQueue:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.capacity = capacity
        self.front = 0                # index of the front element
        self.count = 0                # number of elements

    def is_empty(self):
        return self.count == 0

    def is_full(self):
        return self.count == self.capacity

    def enqueue(self, x):
        if self.is_full():
            raise OverflowError("queue is full")
        rear = (self.front + self.count) % self.capacity
        self.data[rear] = x
        self.count += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("queue is empty")
        x = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return x

    def peek(self):
        return None if self.is_empty() else self.data[self.front]

    def __str__(self):
        return str([self.data[(self.front + i) % self.capacity] for i in
range(self.count)])

q = CircularQueue(5)
for x in "ABCD":
    q.enqueue(x)
q.dequeue(); q.dequeue()
q.enqueue("E"); q.enqueue("F")
print("queue:", q, "| raw array:", q.data, "| front index:", q.front)

```



Why keep `count`?

With only **front** and **rear** indexes, an empty queue and a full queue look the same ($\text{front} == \text{rear}$). Keeping a **count** (or leaving one slot always empty) removes this ambiguity.

5. Part C: Linked-List Queue

Enqueue at the **tail** and dequeue from the **head** of a singly linked list. Both are $O(1)$ and the queue never becomes full.

```
class LinkedQueue:
    class _Node:
        def __init__(self, data):
            self.data, self.next = data, None

    def __init__(self):
        self.head = self.tail = None
        self.count = 0

    def enqueue(self, x):
        node = self._Node(x)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.count += 1

    def dequeue(self):
        if self.head is None:
            raise IndexError("queue is empty")
        x = self.head.data
        self.head = self.head.next
        if self.head is None:
            self.tail = None
        self.count -= 1
        return x

printer = LinkedQueue()
for job in ["CV.pdf", "Assignment.docx", "Timetable.png"]:
    printer.enqueue(job)
while printer.count:
    print("Printing", printer.dequeue())
```

↔ 6. Part D: Deque (Double-Ended Queue)

A **deque** (pronounced “deck”) allows insertion and deletion at **both ends** in $O(1)$. It can act as both a stack and a queue. Python’s `collections.deque` is the standard tool and should be your default queue in Python.

Operation	deque method	Time
Add at rear / front	<code>append(x)</code> / <code>appendleft(x)</code>	$O(1)$
Remove from rear / front	<code>pop()</code> / <code>popleft()</code>	$O(1)$
Look at ends	<code>dq[-1]</code> / <code>dq[0]</code>	$O(1)$
Fixed-size buffer	<code>deque(maxlen=k)</code>	Old items drop automatically
Rotate	<code>rotate(k)</code>	$O(k)$

```
from collections import deque

dq = deque()
dq.append("B"); dq.append("C"); dq.appendleft("A")
print(dq, dq.popleft(), dq.pop(), dq)

last_5 = deque(maxlen=5)           # e.g. last 5 heart-rate readings
for bpm in [72, 75, 80, 78, 90, 95, 70]:
    last_5.append(bpm)
print("buffer:", list(last_5), "average:", sum(last_5) / len(last_5))
```

6.1 Sliding Window Maximum with a Deque

Problem: for every window of k consecutive values, report the maximum. Keep a deque of **indexes** whose values are in **decreasing order**. The front is always the maximum of the current window. Each index enters and leaves once, so the total is $O(n)$.

```
def window_max(nums, k):
    dq, result = deque(), []
    for i, x in enumerate(nums):
        if dq and dq[0] <= i - k:          # front index left the window
            dq.popleft()
        while dq and nums[dq[-1]] < x:      # smaller values can never be a
maximum
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

print(window_max([1, 3, -1, -3, 5, 3, 6, 7], 3))
```

★ 7. Part E: Priority Queue (Preview)

In a **priority queue**, each element has a priority and the element with the **highest priority** (or lowest value) is served first, regardless of arrival time. It is usually implemented with a **heap** (Lab 15). Python provides **heapq**, a **min-heap**: the smallest item comes out first.

```
import heapq

er = []                                # emergency room: lower number =
more serious
heapq.heappush(er, (3, "Ali - fever"))
heapq.heappush(er, (1, "Sara - accident"))
heapq.heappush(er, (2, "Usman - fracture"))
heapq.heappush(er, (3, "Zoya - cough"))
while er:
    priority, patient = heapq.heappop(er)
    print(f"Treating (priority {priority}): {patient}")
```

Structure	Removes	Typical use
Stack	Newest item (LIFO)	Undo, recursion, DFS
Queue	Oldest item (FIFO)	Scheduling, buffering, BFS
Deque	Either end	Sliding windows, palindromes, both
Priority queue	Highest-priority item	Emergency rooms, Dijkstra, task schedulers

8. C++ Corner

C++

```
#include <iostream>
#include <queue>
#include <deque>
using namespace std;

class CircularQueue {
    int* data; int cap, front = 0, count = 0;
public:
    CircularQueue(int c) : cap(c) { data = new int[c]; }
    ~CircularQueue() { delete[] data; }
    bool enqueue(int x) {
        if (count == cap) return false;           // full
        data[(front + count) % cap] = x; count++; return true;
    }
    bool dequeue(int& out) {
        if (count == 0) return false;             // empty
        out = data[front]; front = (front + 1) % cap; count--; return
true;
    }
};

int main() {
    queue<string> q;                               // std::queue: push, pop, front,
back, empty
    q.push("A"); q.push("B");
    cout << q.front() << endl; q.pop();

    deque<int> d = {2, 3}; d.push_front(1); d.push_back(4);
    priority_queue<int> pq;                        // MAX-heap by default in C++
    for (int x : {5, 1, 9}) pq.push(x);
    cout << pq.top() << endl;                      // 9

    CircularQueue cq(3); int v;
    cq.enqueue(10); cq.enqueue(20); cq.dequeue(v); cout << v << endl;
}
```

9. Common Mistakes

Mistake	Effect	Fix
Using <code>list.pop(0)</code> for dequeue	O(n) per dequeue	Use <code>collections.deque</code>
Forgetting modulo in a circular queue	IndexError at the end of the array	<code>(i + 1) % capacity</code>

Mistake	Effect	Fix
Not resetting tail when the linked queue becomes empty	Next enqueue attaches to a dead node	if head is None: tail = None
Confusing front and rear	LIFO behaviour by accident	Enqueue at rear, dequeue at front
Assuming heapq is a max-heap	Smallest comes out first	Push negative priorities for max-first

10. Lab Tasks

Solve all tasks in `Lab11_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Circular Queue with Rear and Resize

Extend `CircularQueue` into `GrowingQueue` that adds a `rear()` method and, instead of raising an error when full, **doubles its capacity** (copy the elements in queue order starting at index 0).

```
GrowingQueue(2): enqueue 1, 2, 3 → capacity 4, front 1, rear 3
```

Task 2 Time to Buy Tickets

People stand in a queue to buy cricket match tickets. `tickets[i]` is how many tickets person *i* wants; each person buys **one ticket per turn** (1 second) and then goes to the back of the line if they need more. Return the time taken for the person at position *k* to finish.

Simulate it with a `deque` of `[index, remaining]` pairs.

```
tickets = [2, 3, 2], k = 2 → 6
```

Task 3 Queue Using Two Stacks

Implement `StackQueue` with `enqueue`, `dequeue`, `peek` and `is_empty` using **only two Python lists used as stacks** (`append/pop()` only).

Hint: push into an `inbox` stack; when the `outbox` stack is empty, pour everything from `inbox` into `outbox`. Each element moves at most twice, so operations are **amortised $O(1)$** .

Task 4 Recent Requests Counter

A web server wants to know how many requests arrived in the **last 3000 milliseconds**. Implement `RecentCounter.ping(t)` which records a request at time *t* (strictly increasing) and returns the number of requests in `[t - 3000, t]`. Use a `deque` and drop old requests from the front.

```
ping(1) → 1, ping(100) → 2, ping(3001) → 3, ping(3002) → 3
```

Task 5 First Non-Repeating Character in a Stream

Characters of a chat message arrive one by one. After each character, report the **first character so far that has appeared exactly once** (or **#** if none). Use a queue of candidates and a count dictionary.

```
"aabc" → "a#bb"
```

Task 6 Bank Counter Simulation

Simulate a bank with **one cashier**. Customers arrive at given minutes with given service times: `customers = [(arrival, service), ...]` sorted by arrival. The cashier serves in FIFO order.

Return `(average_wait, max_queue_length)`, where `wait` = start of service – arrival (rounded to 2 decimals) and the queue length counts customers **waiting** (not the one being served). Keep a queue of the service start times of waiting customers.

```
[(0, 5), (1, 3), (2, 1), (10, 2)] → waits 0, 4, 6, 0 → (2.5, 2)
```

Task 7 Sliding Window Maximum (Traffic Monitoring)

A traffic camera records cars per minute. For each window of `k` minutes, report the **maximum**. Implement `window_max_k(counts, k)` in $O(n)$ with a deque of indexes, and verify it against a brute-force version on random data.

```
[1, 3, -1, -3, 5, 3, 6, 7], k = 3 → [3, 3, 5, 5, 6, 7]
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab11` folder.

#	Problem	Level	Concept practised
933	Number of Recent Calls	Easy	Deque window (Task 4)
2073	Time Needed to Buy Tickets	Easy	Queue simulation (Task 2)
232	Implement Queue using Stacks	Easy	Two stacks (Task 3)
225	Implement Stack using Queues	Easy	Reverse thinking
1700	Number of Students Unable to Eat Lunch	Easy	Queue simulation

#	Problem	Level	Concept practised
622	Design Circular Queue	Medium	Modulo arithmetic
239	Sliding Window Maximum	Hard	Monotonic deque (Task 7)



12. Home Task

1. Finish all tasks and push `Lab11_Student.ipynb` to GitHub.
2. Implement a **deque from scratch** using a circular array with `append`, `appendleft`, `pop` and `popleft`.
3. Extend the bank simulation to **two cashiers** and compare average waiting time for the same customers.
4. **Mini-project: Hospital Emergency System.** Patients arrive with a severity (1–5). Use `heapq` so that the most severe patient is treated first and, among equal severity, the one who arrived first (hint: push `(severity, arrival_number, name)`). Print the treatment order and the average waiting time.
5. Solve at least 3 LeetCode problems from the list.

? 13. Viva Questions

1. What is FIFO? How is a queue different from a stack?
2. List the queue operations and their time complexities.
3. Why is `list.pop(0)` a bad way to dequeue?
4. What problem does a circular queue solve?
5. How do you compute the next index in a circular queue?
6. How do you distinguish a full circular queue from an empty one?
7. What is a deque? Name two uses.
8. What is a priority queue? How is it different from a normal queue?
9. How can you build a queue from two stacks? What is the amortised cost?
10. Which graph algorithm uses a queue?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- Python `queue`, `collections.deque` and `heapq` documentation: docs.python.org/3/library
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.